# Project - Submission Sheet

Software Testing - Random Testing

**Name:** Arpita Sarkar
**Group:** 4

**Software Testing** - Summer 2024 - May 11, 2025 - Sibylle Schupp / Daniel Rashedi

## Task 1 - Random Testing

a) Add the 4 generated Python tests of each group member to your repository in the corresponding folder. Add a README.md file that explains how to build / run the tests.

b) Provide screenshots of your terminals that indicate your execution of the new tests:

*Answer:*

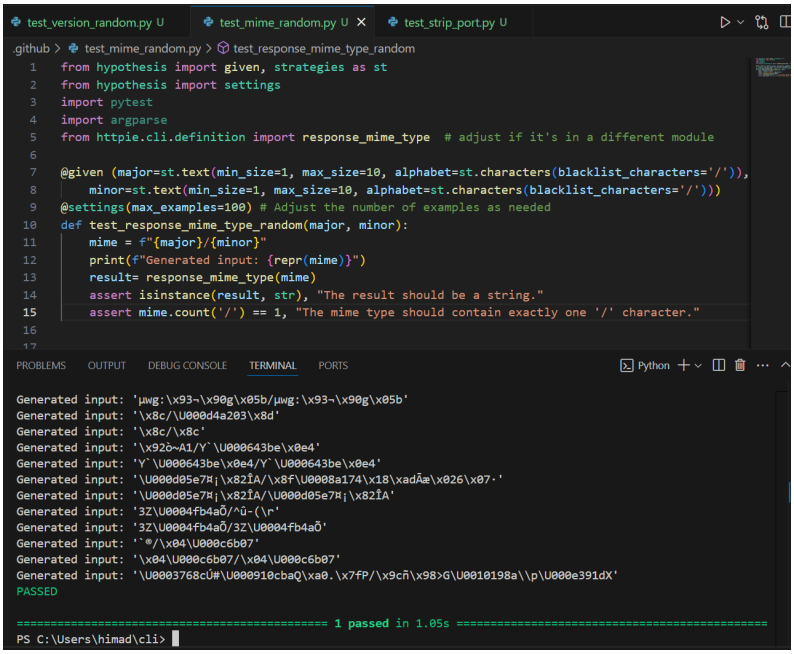


Figure 1: test outcome of function `response_mime_type`

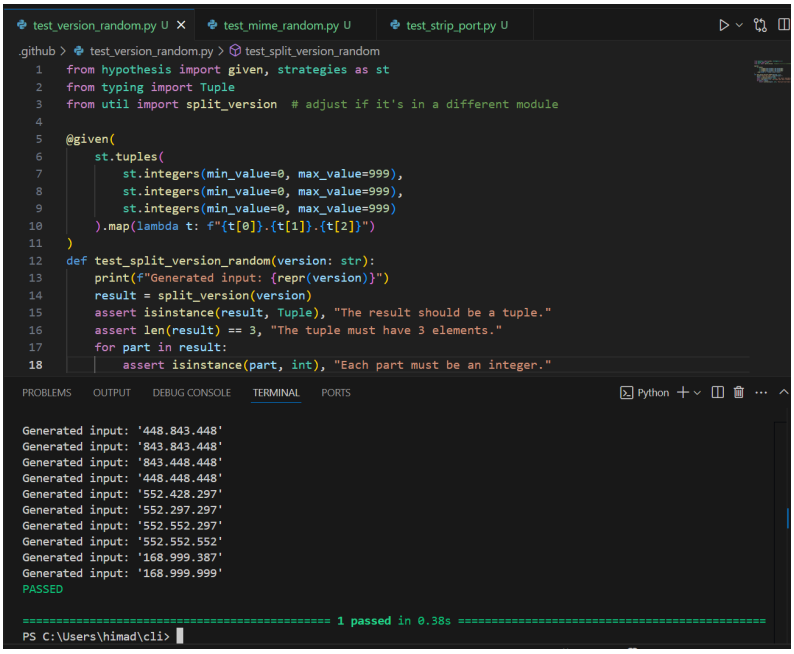


Figure 2: test outcome of function `Split_version`

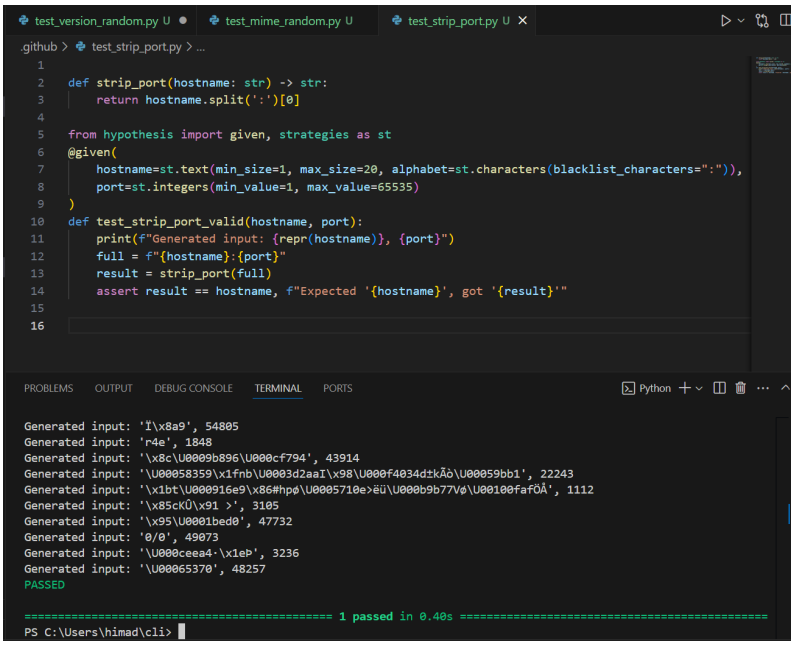


Figure 3: test outcome of function `Strip_port`

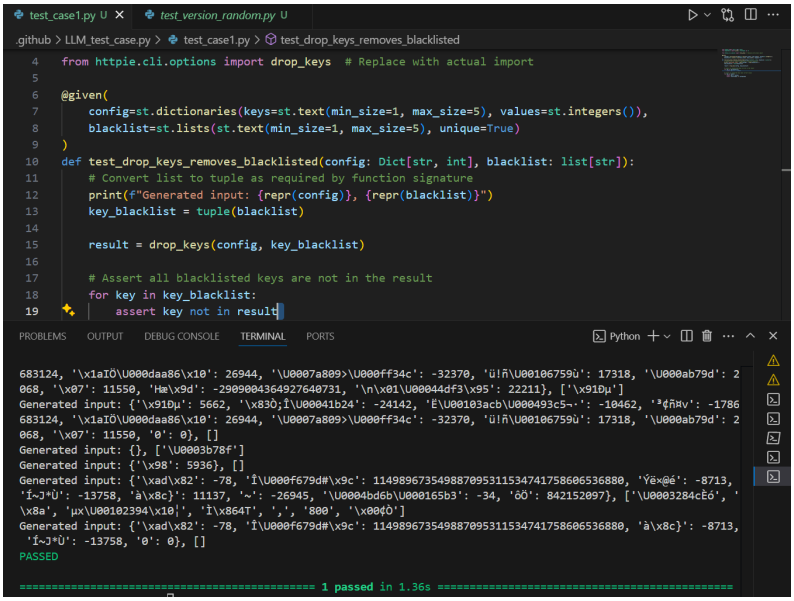


Figure 4: test outcome of function `Drop_keys`

c) Assess the input values generated by **`hypothesis`**. Do the tests represent meaningful scenarios?:

*Answer:*

**Test of `response_mime_type`**: The function aims to take string input and allows to pass the input only with one $'/'$.The test inputs generated by Hypothesis are structured to simulate valid MIME type strings in the format "type/subtype", such as "application/json".This design makes the inputs both syntactically valid and semantically meaningful for the function, closely mirroring real-world scenarios where MIME types are used in HTTP headers or API responses. Therefore, the test inputs represent realistic and relevant cases for validating the function's correctness.

**Test of `Split_version`**:This function takes input as string of version like '1.2.3' and generates output as a tuple of interger values as (1,2,3). We used strategies for hypothesis as integer.Then converted it to a string. The inputs generated by Hypothesis represent meaningful and realistic scenarios for testing the function.It ensures test reflects practical and expected usage. By using structured numeric values and controlling the format, the test avoids irrelevant or malformed input and focuses on verifying the correctness of parsing well-formed version strings, making the test both robust and semantically relevant.

**Test of `Strip_port`**:The test's hypothesis-generated input values are significant because they faithfully capture the appearance of hostnames with ports in actual networking situations, like localhost:8080 or example.com:443. In order to guarantee that the sole colon in the created input string originates from the hand insertion between the hostname and port, the hostname is formed as a non-empty string without the ':' character. For use cases such as URL parsing or socket connections, the complete input is feasible because the port is limited to the acceptable range for TCP/UDP ports (1–65535). The test is useful for verifying expected behavior in real-world applications because of its design, which makes sure it closely resembles the usual input formats the function would process.

**Test of `drop_keys`**:The function returns a new dictionary by removing all keys listed in the **`key_blacklist`** from the given configuration dictionary. It is useful for filtering out unwanted or sensitive keys before processing or sharing data. The hypotheses test is meaningful for the function. It considers edge cases and also generate input randomly that is valid for the function.

## AI Review Task

Assess the correctness and understandability of the LLM's generated tests applying the paper's methodology. Add any code relevant for your assessment to the answer:

*Answer:*

[Group:4 LM studio]

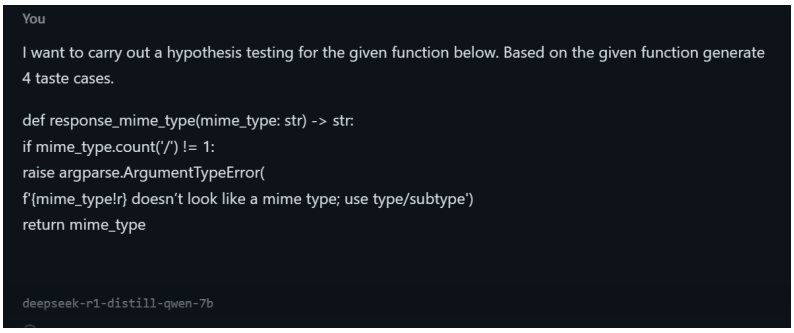


Figure 5: Prompt

[Group:4 Assessment]

```
    import argparse
from httpie.cli.definition import response_mime_type

def test_valid_mime_type_text_csv():
    mime = "text/csv"
    result = response_mime_type(mime)
    assert result == "text/csv"
```

Listing 1: Valid main type with subtype

```
import argparse
from httpie.cli.definition import response_mime_type
import pytest
def test_valid_mime_type_text_csv():
    mime = "image/png/"
    mime_type = response_mime_type(mime)
    assert mime_type == mime
```

Listing 2: Invalid Multiple Slashes

```
        import argparse
from httpie.cli.definition import response_mime_type
import pytest
def test_valid_mime_type_text_csv():
    mime = "image"
    mime_type = response_mime_type(mime)
    assert mime_type == mime
```

Listing 3: Missing slash

```python

import argparse
from httpie.cli.definition import response_mime_type
import pytest
def test_valid_mime_type_text_csv():
    mime = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
    mime_type = response_mime_type(mime)
    assert mime_type == mime
```

Listing 4: Complex Mime Type with One Slash

**Test case scenarios:**

- 1st case: The input is valid and follows a valid mime format such as 'text/ csv'.
- 2nd case: Input is invalid with because it contains more than one /.
- 3rd case:Input is also invalid as it contains no /.
- 4th case: Input is valid but more complex.

**Comparison with LLM test cases:**

- 1st test case identifies if the function is able to find valid input.The motive of our test case was same.
- 2nd test case is also relevant with our test as it checks if there is exactly one backslash in the input.
- 3rd test case is not fully matched with our test case.In our test case we did not consider about the empty backslash condition. But this situation is also important for checking the validity of the function.
- 4th test case also aligns with out test goal. It checks if the function is able to handle more complex input successfully.

**Ares to improve:** If function behavior isn't fully documented, AI might assume too much. There is no integration feedback and runtime awareness provided by AI.

**Conclusion:** The four test cases generated by LM studio is syntactically correct and logically aligned with the function's intent.To ensure test correctness we also need to think the edge cases logically to avoid any mistake.